

Квантизация модели.

После обучения получаем модель в формате pytorch с весами в обычном FP32/FP16. И для LogBERT это примерно 263 Мбайт. И дело тут не столько в размере, а в том, что такая модель требует для своей работы GPU.

Выбор формата квантизации.

Существуют разные способы квантизовать, "облегчить" модель путем перевода или всех, или части весов в INT8, или даже в 4 битное и ниже представление. Так как модель относительно маленькая, то не стоит увлекаться в этом процессе и достаточно выбрать INT8. И если ограничиться им то возможны два варианта.

1. Dynamic Quantization (int8, линейные слои)

Когда удобно: самый простой вход в квантизацию для CPU-инференса.

- Что квантизируем: `nn.Linear` (и иногда LSTM/GRU, если они есть). В BERT это основная масса параметров.
- Тип: веса в `int8`, активации остаются в float (или pseudo-int8 на лету).
- Плюсы:
 - Минимум кода.
 - Почти не трогает архитектуру.
 - Поддерживается "из коробки" в PyTorch (`torch.quantization.quantize_dynamic(...)`).
 - Хорошо работает на x86 с MKL/oneDNN.
- Минусы:
 - Выигрыш ~2–4× по памяти и ускорению, но не радикальный как 4-бит.
 - Не всё квантизируется (embeddings, attention softmax и т.п. остаются в float).

Практически:

1. Обучил LogBERT в обычном FP32/FP16.
2. Сохранил state_dict.
3. Для CPU-инференса — загружаешь модель, вызываешь `quantize_dynamic`, сохраняешь отдельный чекпоинт для CPU-версии.

2. ONNX Runtime + квантизация

Если хочется **универсальный формат** (deploy в разные сервисы, языки, вне Python), то:

1. Экспортируешь LogBERT в ONNX.
2. Применяешь:

- Dynamic quantization (weights int8).
- Static quantization (weights+activations int8) через оптимизаторы ONNX Runtime.

3. Гонишь inference через ONNX Runtime (CPU Execution Provider).

Плюсы:

- Легко отдавать модель другим сервисам/языкам.
- Нормальная производительность на CPU (особенно с оптимизациями для AVX2/AVX-512).

Минусы:

- Нужно аккуратно описывать входы/outputs (маски, позиции и т.п.).
- Придётся однажды покопаться в экспорте attention-масок, если есть кастомные фишки.

Вывод.

Выбираем **ONNX**, так как очень важно следующее:

- Легко отдавать модель другим сервисам/языкам. Значит можно легко упаковать в контейнер.
- Нормальная производительность на CPU (особенно с оптимизациями для AVX2/AVX-512).

Квантизация.

Выполняется скриптом `15-export_logbert_to_onnx.py`, который загружает существующую модель как `model/checkpoints/model.pt`, затем словарь из `dataset/vocab/event_vocab.json` и выполняет преобразование. Сначала в FP32 ONNX-модель, а затем делает int8-квантизацию через ONNX Runtime.

Естественно, это все требует установки дополнительных пакетов в рабочее окружение.

```
> pip freeze | grep onnx
onnx==1.19.1
onnx-ir==0.1.12
onnxruntime==1.23.2
onnxruntime-tools==1.7.0
onnxscript==0.5.6
```

Неожиданная проблема.

Главной сложностью оказалось падение процесса в ходе исполнения int8-квантизации. Сообщение об ошибке следующее:

```
Inferred shape and existing shape differ in dimension 0: (512) vs (45950)
```

Выглядит так, что где-то в процессе инференса **ONNX Runtime** путает `max_len=512` с `vocab_size=45950` (классическая история для моделей с большими embedding-матрицами).

Решить эту проблему оказалось возможным лишь путем подмены форварда в модели. Исходно модель описана как класс **UnifiedLogBERT**. И внутри определен форвард:

```
def forward(self, input_ids: torch.Tensor, sec_ids: torch.Tensor) ->
    torch.Tensor:
    x = self.token_emb(input_ids)                # (B,L,D)
    s = self.section_emb(sec_ids).unsqueeze(1)    # (B,1,D)
    x = x + s
    x = self.pos(x)
    h = self.encoder(x)                          # (B,L,D)
    h = self.norm(h[:, -1, :])                  # last position → next-event
    output: torch.Tensor = self.lm_head(h)       # (B,V)
    return output
```

Этот форвард абсолютно правильный и в обучении и в инференсе pytorch модели давал ошибку при проведении shape inference по графу в процессе квантизации. Правильным вариантом форварда для безошибочной квантизации будет иной:

```
def forward(self, input_ids, sec_ids):
    # (B, L, D)
    x = self.base.token_emb(input_ids)
    s = self.base.section_emb(sec_ids).unsqueeze(1)
    x = x + s
    x = self.base.pos(x)
    h = self.base.encoder(x)                    # (B, L, D)
    h = self.base.norm(h)                      # (B, L, D)
    logits_all = self.base.lm_head(h)          # (B, L, V)
    logits = logits_all[:, -1, :]              # (B, V)
    return logits
```

Но таким путем мы если не ломаем фатально существующую модель, то сильно её замедляем. И чтобы бесконфликтно решить это был создан второй класс **UnifiedLogBERTForONNX** с новым форвардом, который и использовался в процессе трансформации.

Выглядит это так. Сначала загружаем исходную модель из чекпойнта:

```
base_model = load_model_from_checkpoint(
    config=config,
    checkpoint_path=checkpoint_path,
    vocab_size=vocab_size,
    device=torch.device("cpu"), # Для экспорта используем CPU
    eval_mode=True,
)
```

А потом меняем ей тип:

```
model = UnifiedLogBERTForONNX(base_model)
model.eval()
```

И далее во всех преобразованиях начинает вызываться уже новый форвард. Это изменение касается только скрипта преобразования в ONNX формат.

Артефакты.

В результате преобразования были созданы следующие артефакты:

```
> pwd
model/onnx
> ll
total 324M
-rw-rw-r-- 1 alekseybb alekseybb 559K Nov 20 02:42 logbert_fp32.onnx
-rw-rw-r-- 1 alekseybb alekseybb 257M Nov 20 02:42 logbert_fp32.onnx.data
-rw-rw-r-- 1 alekseybb alekseybb 66M Nov 20 02:42 logbert_int8.onnx
-rw-rw-r-- 1 alekseybb alekseybb 292 Nov 20 02:42
onnx_export_metadata.json
```

logbert_fp32.onnx (559K) + logbert_fp32.onnx.data (257M)

Это одна модель в формате **ONNX с external data**:

- **logbert_fp32.onnx**
 - содержит *структуру* графа:
 - входы (**input_ids**, **sec_ids**);
 - выход (**logits**);
 - список нод (MatMul, Add, LayerNorm, Embedding, etc.);
 - метаданные (opset, producer_name, возможно атрибуты quantization-прохода).
 - сами веса (**weights**, **biases**, embedding-матрицы) в нём **не лежат**, там лишь ссылки на них.
- **logbert_fp32.onnx.data**
 - содержит **сырые тензоры весов FP32**:
 - **token_emb.weight** (размер [**vocab_size**, **d_model**]);
 - **section_emb.weight** ([**section_buckets**, **d_model**]);
 - веса/биасы всех слоёв **TransformerEncoder**;
 - **lm_head.weight** ([**vocab_size**, **d_model**]) и т.п.
 - ONNX связывает их по имени/offset'ам.

Это сделано потому, что “монолитный” ONNX с такими весами легко переваливает за 2GB, а формат по умолчанию это не любит, поэтому PyTorch разбивает на **.onnx** + **.onnx.data**.

logbert_int8.onnx (66M)

Это **квантизованная** версия той же модели:

- веса некоторых / всех линейных слоёв и эмбеддингов приведены к `int8` (типично `QInt8`);
- в графе появляются дополнительные ноды:
 - `QuantizeLinear`, `DequantizeLinear`,
 - иногда `MatMulInteger`, `QLinearMatMul`, `QLinearConv` и т.п.
- структура входов/выходов **должна совпадать** с FP32: всё тот же `input_ids`, `sec_ids` → `logits`.

Размер 66M говорит, что квантизация сработала: int8-веса занимают ~4x меньше места, плюс часть структур сохранилась в float.

onnx_export_metadata.json (292 байта)

Это маленький служебный файл. Он просто фиксирует:

- откуда брали веса,
- куда сохранили FP32 и int8 модели,
- с какими `max_len` и `opset` экспортировали.

Полезно для воспроизводимости и отладки.

```
> cat onnx_export_metadata.json | jq
{
  "export_time": "2025-11-19T23:42:28.747638Z",
  "checkpoint_path": "model/checkpoints/model.pt",
  "onnx_fp32_path": "model/onnx/logbert_fp32.onnx",
  "onnx_int8_path": "model/onnx/logbert_int8.onnx",
  "max_len": 512,
  "opset": 18
}
```

Проверочный инференс.